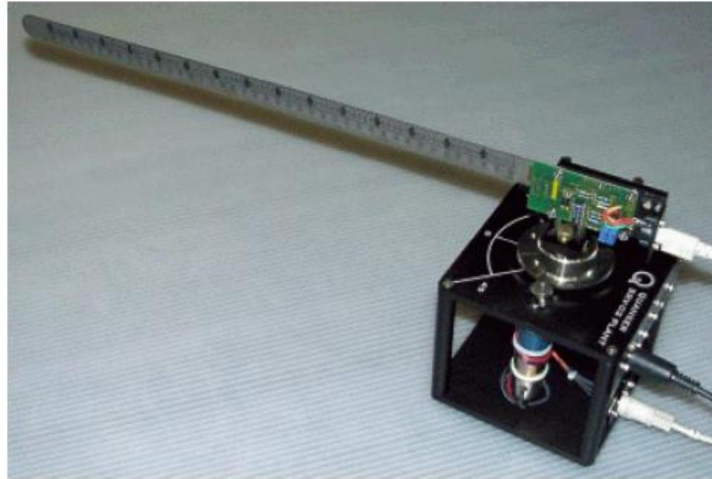


Control of Cyber-physical Systems

LABORATORY WORK

Identification and Computer Control of a Flexible Robot Arm Joint



Prepared by:

Pedro dos Santos

Sérgio Pequito

João Pedro Gomes

Alexandre Bernardino

Alberto Vale

João Miranda Lemos

2026/2027 – 1st Semester

Instituto Superior Técnico
Department of Electrical and Computer Engineering
Scientific Area of Systems, Decision, and Control

Read this with attention!

What you need to bring and do at the laboratory?

1. You must bring to the lab a flash drive to store the results obtained during the class sessions;
2. When using the lab computers make sure that you are using a working folder with write permissions;
3. The working folder with all the Matlab/Simulink project files and the experimental data must be saved inside the computer's disk and regularly backed up to the external flash drive.
4. Avoid saving files directly to the flash drive, as there have been instances where files became corrupted, leading to the loss of work.

At the lab: Safety warnings!

1. To avoid being injured, all the students should be at a safe distance from the flexible bar to avoid being hit if it turns.
2. When performing tests, one student must always have a finger close to the power amplifier switch to stop the bar if it starts running.
3. In order to avoid malfunctions, never allow the bar to rotate over itself.

List of symbols and units

- D [m] → Distance of deflection of the bar.
- L [m] → Bar length.
- θ [degree] → Angle of rotation of the motor shaft with respect to a "zero" direction.
- α [rad] or [degree] → Angle of deflection of the flexible bar with respect to the motor shaft angular direction.
- ω [degree/s] → Motor angular speed.
- θ_e [V] → Electrical tension yielded by the sensor of θ
- α_e [V] → Electrical tension yielded by the sensor of α .
- K_b [degree /V] → Flexible bar deflection transducer constant.
- K_p [degree /V] → Motor shaft angular position transducer constant.
- y [rad] → Total angular position of the tip of the flexible bar tip with respect to the "zero" direction. $y = \theta + \alpha$.

Translation to Portuguese of selected words

- Arm - braço
- Comb - pente
- Gear box - caixa de desmultiplicação
- Motor shaft - veio do motor
- Plant - instalação (a controlar)
- Strain gage - extensómetro
- Tip - ponta

Contents

1	Objective	5
	Organisation of the laboratory sessions	5
2	Plant description	8
	 Part I – Q1 Key considerations for control system design	8
2.1	Measuring the bar deflection	9
2.2	Flexible bar deflection transducer	11
2.3	Shaft angle transducer	12
3	Software for data acquisition and control	12
3.1	Simulink	13
	 Tasks to be performed in sessions L2–L3	14
3.2	Generation, viewing and recording signals.	15
3.3	Sending commands to the motor.	16
3.4	Reading signals from the sensors.	17
3.5	Sensor Calibration.	17
	 Part I – Q2 Interfacing the plant with the computer	18
4	Plant model identification	19
	 Tasks to be performed in sessions L4–L7	23
	 Part I – Q3 The plant model	25
5	Controller design	26
5.1	LQ control law	27
5.2	Kalman filter	28
5.3	Loop transfer recovery (LTR)	29
5.4	LQG controller structure	29
	 Tasks to be performed in sessions L8–L12	29
5.5	Documentation	31
	 Part II – Q4 The control design	31
6	Instruction on the report and software to develop	33
6.1	Report Content and Submission	33
6.2	Language	33
6.3	Report Format	33
6.4	Answer Guidelines	34
6.5	Submission	34
6.6	Originality and Plagiarism	34

1 Objective

This work focuses on the identification and computer control design for positioning a flexible joint in a robot arm (represented by a flexible bar). The project exemplifies the application of Control to Cyber-Physical Systems, where a computer interfaces with a physical plant (the flexible robot arm joint) to modify its behavior through computational algorithms.

The work is divided into two parts:

- **Part I – System Identification:** In this phase, we develop a plant model that correlates the motor driving the arm with the angular position of its tip. This model is derived using System Identification methods and data collected from experiments conducted on the physical plant.
- **Part II –Controller Design and Implementation:** Using the model obtained in Part 1, we design a controller to position the tip of the robot joint at a desired angle. The controller is then implemented and tested on the actual plant to validate its performance.

This two-part approach allows us to explore the full cycle of control system design, from modeling to implementation, in the context of a practical robotic application.

Organisation of the laboratory sessions

The laboratory runs in weekly sessions of **1h30** each. Every shift has thirteen sessions, denoted **L1** to **L13** throughout this guide. Because the Monday shifts do not meet in week 5 (public holiday) and the Thursday shifts do not meet in week 7, the two groups of shifts fall on different calendar weeks between L5 and L6, and realign from L7 onwards. Table 1 gives the correspondence.

Session	L04/L05 (Thu)	L02/L03 (Mon)	Focus
L1	week 1	week 1	Group formation, safety briefing, walk-through of this guide
L2	week 2	week 2	Simulink, signal generation, D/A, first motor command (Sec. 3.2–3.3)
L3	week 3	week 3	A/D, sensor readings, calibration of K_p and K_e (Sec. 3.4–3.5)
L4	week 4	week 4	Design of the excitation signal and collection of identification data
L5	week 5	week 6	Data processing and ARMAX identification, model order selection
L6	week 6	week 7	Integrator, state-space conversion, model validation
L7	week 9	week 9	Review of the identification: filter pole, orders, sampling interval
L8	week 10	week 10	LQ design on the identified model, root-square locus in R
L9	week 11	week 11	Kalman filter and LTR, closed-loop simulation of the LQG
L10	week 12	week 12	Real-time LQG diagram, pre-filter, first tests on the plant
L11	week 13	week 13	Tuning on the plant: effect of R and R_v
L12	week 14	week 14	Final closed-loop measurements. Report due 23:59 on the day of this session
L13	week 15	week 15	Final laboratory assessment: collective and individual validation questions, in the group's own shift slot

Table 1: Correspondence between the thirteen 1h30 laboratory sessions and the tasks of this guide.

Twelve of the thirteen sessions are working sessions, and eleven of them (L2–L12) precede the report deadline, for a total of 16h30 at the bench. L13 is not a working session: it is the final laboratory assessment on the report that was submitted at the end of L12. Bench time is still shorter than the guide assumed in previous years, so the preparation asked for before each block of sessions is no longer optional: sessions in which the MATLAB scripts still have to be written from scratch will not be enough to complete the tasks.

The laboratory is deliberately sequenced against the lecture course, so that each block of sessions asks for material that has already been treated in class. Table 2 makes that correspondence explicit. Preparing for a MAP30 and preparing for the following laboratory sessions are, to a large extent, the same activity.

Session	What the session asks you to do	Module it relies on	When it is covered
L2	Signal generation, real-time configuration, D/A	M1 – discrete-time signals, difference equations, sampling	discussed week 2, MAP30 week 3
L3	Sensor readings; estimation of K_p and K_e by least squares	M3.1 – parameter estimation fundamentals	discussed week 4 (<i>L3 runs one week ahead: watch M3.1 before the session</i>)
L4	Excitation signal design, sampling at 0,02 s, data collection	M2.2 – sampling and discretization; M3.7 – complete system identification	discussed weeks 2 and 4
L5	Detrending, differentiation, filtering, ARMAX estimation	M3.1–M3.6 – estimation, RLS, stochastic processes, maximum likelihood	discussed week 4, MAP30 week 5
L6	Integrator, conversion to state space, validation	M2.1 – from difference equations to state space; M3.8	discussed weeks 2 and 4
L7	Model order review, open-loop pole-zero and frequency response	M1.3 – discrete transfer functions and frequency response	discussed week 2, MAP30 week 3
L8	dlqr, root-square locus in R , closed-loop eigenvalues	M4 – state feedback, LQR, dynamic programming	discussed week 6, MAP30 week 9
L9	dlqe, LTR, simulation of the full LQG loop	M5 – uncertainty models, state estimation, Kalman filtering	discussed week 10, MAP30 week 11
L10	Real-time LQG diagram, pre-filter, first tests on the plant	M6.1–M6.3 – separation principle, LQG, its DP solution	discussed week 12, MAP30 week 13
L11	Tuning R and R_v on the plant	M6.4 – LQG cost analysis, infinite horizon, practical applications	discussed week 12
L12	Closed-loop frequency response and loop gain	M1.3 and M6.4	discussed weeks 2 and 12

Table 2: Correspondence between the laboratory sessions and the lecture modules on which they depend.

Two points are worth singling out. First, **L3 is the one session that runs ahead of the lectures**: the calibration constants are estimated by least squares, which is treated in M3.1, discussed only in week 4. The fit involved is an elementary one, but you should watch M3.1 before L3 rather than after it. Second, **L9 and L10 fall in the same week as the discussion session of the module they use** – M5 in week 10 and M6 in week 12. Under the flipped format the videos are meant to be watched before the discussion, so this is not a conflict, but a group that postpones the videos to the week of the MAP30 will arrive at those two sessions unable to use them.

2 Plant description

The system under study is a flexible robot arm comprising a single joint. This joint is actuated by a DC motor, which is powered by a dedicated amplifier. The motor's shaft is rigidly connected to one end of a flexible bar, forming the joint mechanism. Figure 1 presents an image of this physical setup, referred to as the "plant," which is the focus of the control system.

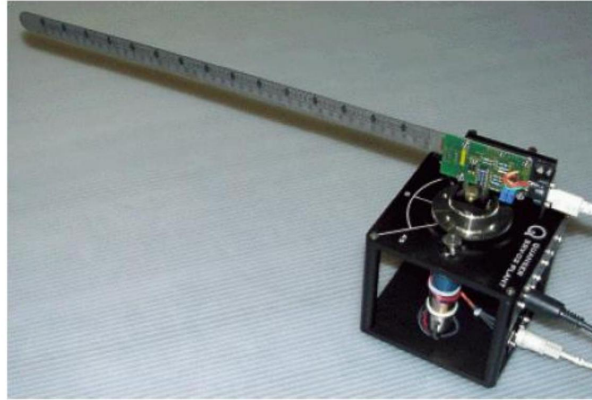


Figure 1: The physical system (plant) to control.

The *control objective* is to manipulate the motor so that the tip of the flexible bar precisely follows a specified angle.



Part I – Q1 Key considerations for control system design

- Why is achieving this control objective nontrivial?
- Is it feasible to create a lookup table of electric voltages for the DC motor that ensures the bar tip remains at a fixed position?
- What would be the result if a constant voltage is applied to the motor?
- Would a simple feedback controller, consisting of a single gain amplifying the tracking error, effectively address the problem?
- **Reflect on the dynamics of the motor and the bar.** What might the approximate open-loop pole distribution of the system look like?



The next figure, Figure 2, presents a schematic diagram illustrating the interconnection between the plant comprising the power amplifier (PA), the DC motor with its gearbox (M), the flexible bar, and the computer.

The computer acquires the angular position of the flexible bar's tip – computed as the sum of the motor shaft's angular position and the bar's deflection – via the A/D converters. At each sampling interval, a computer algorithm processes this data and determines the appropriate electrical voltage to be applied to the motor. This voltage is then transmitted through a D/A converter to the power amplifier (PA), which drives the motor.

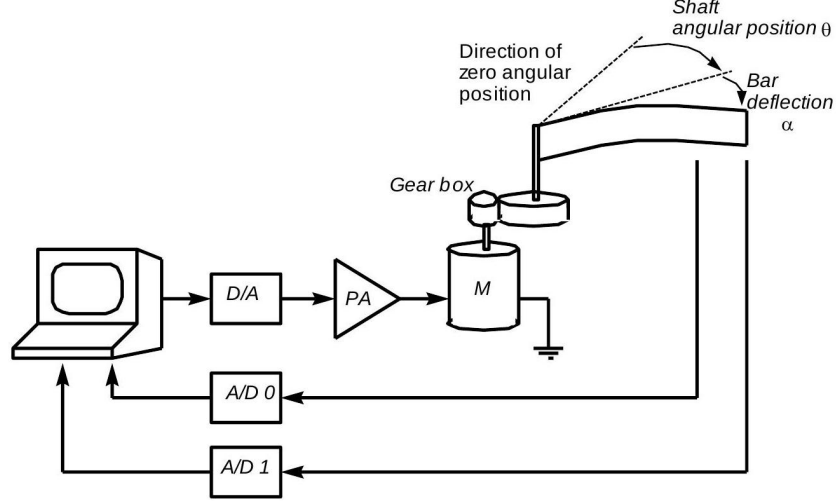


Figure 2: Schematic diagram of the plant’s hardware interconnected with the computer control system. The motor M (including a gearbox) is driven by the output of a power amplifier PA and actuates the flexible bar. The computer interfaces with the plant using A/D and D/A converters.

2.1 Measuring the bar deflection

The flexible bar is equipped with a strain gauge (Figure 3) to measure its deflection. This strain gauge operates on the principle of changing electrical resistance; as the bar deflects, the strain gauge undergoes slight stretching on one face and compression on the other. These deformations cause variations in the resistance, which are approximately proportional to the deflection at the tip of the bar. For more detailed information on strain gauge sensors, please refer to the following link:

<http://www.omega.com/literature/transactions/volume3/strain.html>.

Figure 3 illustrates a close-up of the flexible bar, highlighting the strain gauge’s mounting, indicated by the red encirclement.

With respect to the circular movement of the motor and the tip of the flexible bar, sketched in Figure 4, consider the following quantities:

- L [m], the bar length;
- D [m], the length of the deflection of the bar tip;
- α [rad], the angular deviation of the bar tip with respect to the motor shaft direction;
- θ [rad], the angle of the motor shaft direction with respect to a “zero” direction.

Recall from basic geometry that the value of an angle in radian (rad) is given by the length of the corresponding arc of a circumference of radius of length to 1 in the length unit used. Consequently, the angular deflection of the bar tip, in relation to its linear deflection, can be approximated by the following expression:

$$\alpha = \frac{D}{L} \quad (\text{rad}).$$



Figure 3: Close-up of the flexible bar, illustrating the strain gauge's mounting (highlighted by the red encirclement). The electronic circuit located at the fixed end of the bar includes a Wheatstone bridge, which is employed to measure the electrical resistance of the strain gauge.

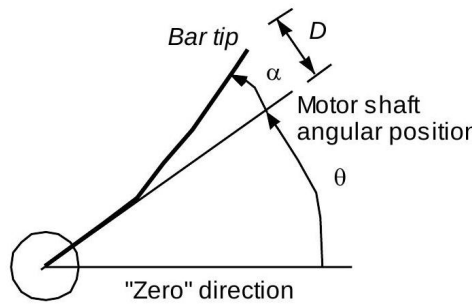


Figure 4: Schematic model of the flexible bar deflection.

In practice, it is preferable to work in (degree) instead of (rad) units. For that sake, observe that the conversion between the same measure expressed in these units is made by

$$\alpha_{[\text{degree}]} = \frac{180}{\pi} \alpha_{[\text{rad}]}.$$

Hereafter, we will always use the angles expressed in degrees.

The total angle that defines the position of the tip of the flexible bar is

$$y = \theta + \alpha \quad (\text{degree}).$$

Historical Curiosity

Did you know? The importance of using consistent units cannot be overstated. In one infamous incident, a space probe bound for Mars was lost because one part of the design team used centimeters for length measurements, while another used inches. This discrepancy led to the failure of the mission.

2.2 Flexible bar deflection transducer

The angle sensors provide an electric tension that is an image of the corresponding angle. In the case of the flexible bar, the sensor provides an electrical tension α_e that is related to the actual deflection angle α by

$$\alpha = K_e \alpha_e,$$

where K_e is a constant with units [degree/V]. Figure 5 shows a block diagram that relates the physical variable α (angle) with its electrical tension image α_e .

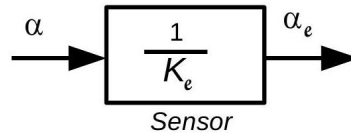


Figure 5: Relation between one angle and its measure given by the sensor.

To estimate the value of the constant K_e , a series of experiments is conducted. Each experiment involves deflecting the bar by a known amount and recording the corresponding electrical voltage.

For these experiments, the bar is securely mounted on a support stand, as illustrated in Figure 6. At the end of the stand, there is a calibration “comb” with several slots spaced at intervals of $\frac{1}{4}$ inch (1 inch = 2.54 cm), which, as shown in Figure 7, enables precise deflection of the bar by predetermined amounts.

In the “ideal” situation, the electrical tension yielded by the sensor would be exactly proportional to the deflection angle. In practice, this is not true because:

- The relation is not exactly linear;
- There are small random fluctuations from experiment to experiment.

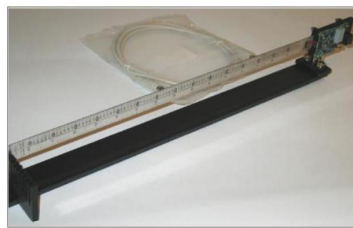


Figure 6: The flexible bar mounted on its calibration rig.

For these reasons one should:

- Characterize the deviations with respect to the linear behaviors;
- Make several experiments.

Observation: It is remarked that the linear dependency of electrical tension on deflection is only expected for relatively small deflections. Large deflections may cause a permanent deformation of the bar and should of course not be done.

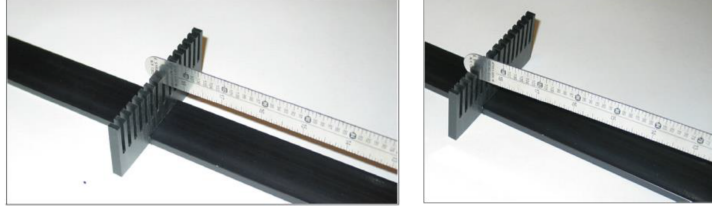


Figure 7: The calibration “comb” for two different bar deflections: In rest, with no deflection (left) and deflected to the left by one slot (right).

Due to the deviations with respect to the ideal linear behavior, the value of the constant K_e corresponds to an average behavior. As such, this constant is estimated from the set of data obtained in the different experiments using the least squares algorithm.

The way to read the electrical quantities will be explained below in detail.

2.3 Shaft angle transducer

The shaft angle is measured using a rotary potentiometer, with its axis rigidly connected to the motor shaft. Figure 8 illustrates the internal structure of the potentiometer. A cursor (point 3), which is firmly attached to the potentiometer axis (point 4) and thus to the motor shaft, moves along a coiled resistive wire (point 1). As the cursor moves, the resistance changes proportionally to the angle of rotation. This varying resistance is then converted into an electrical voltage by the circuit. The resistive element is powered by a voltage of +5 V at one end (point 5) and -5 V at the other end (point 9), resulting in an output voltage that ranges between -5 V and +5 V, depending on the cursor’s position.

It is remarked that, since there are no mechanical stops at the extremes of the resistance (end of course), the measured voltage repeats itself every whole rotation of the cursor.

Therefore, the shaft angle θ is approximately given as a function of the sensor output θ_e by

$$\theta = K_p \theta_e,$$

where K_p is a constant with units [degree/V].

It is also remarked that, due to the coiled structure of the resistance, there is a quantization effect that causes the resistance to vary in a staircase form when the angle varies.

Before proceeding to the estimation of the transducer constants from plant data, the use of the data acquisition and control software is explained.

3 Software for data acquisition and control

To build a plant model for controller design we need to perform experiments on the plant and record data from them. Model parameters are then estimated from these data.

To carry out data acquisition (reading data from the plant sensors connected to the A/D converters) and process control (sending commands to the actuator, connected to the D/A converter) the software “MATLAB” + “Simulink” with toolboxes will be used.

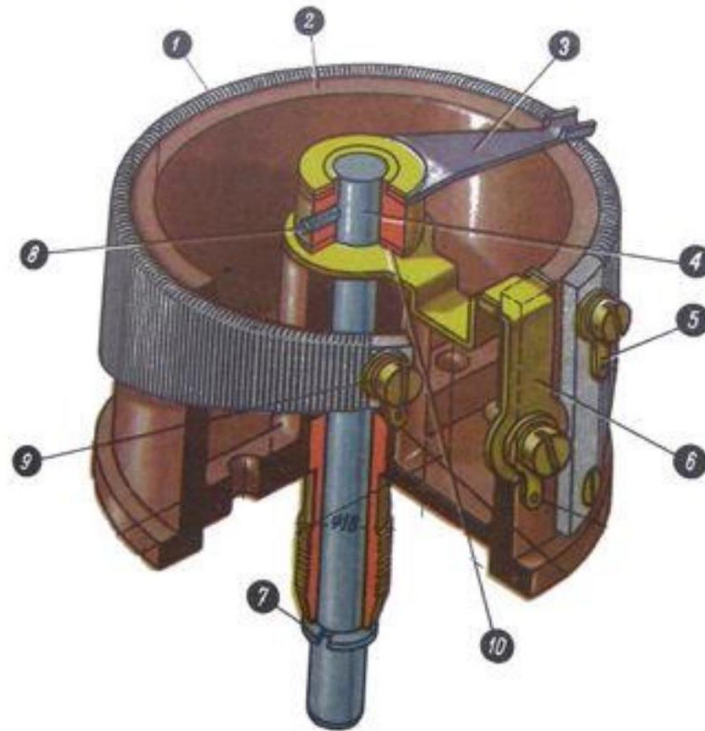


Figure 8: The internal scheme of the potentiometer used to measure the shaft angle.

With “Simulink” it is possible to design data acquisition and control systems through a simple graphical interface and with plenty of features.

Although not essential to complete this work, the student is advised to consult the manuals of these software tools, made available on the website of the course. In the following lines, a brief description of the software to be used is presented.

It is assumed that the student has a basic knowledge of MATLAB and Simulink, that can be easily acquired either in the previous course on Modelling and Simulation or by following one of the many tutorials available in the Web.

3.1 Simulink

Simulink is a simulation environment that works in conjunction with MATLAB and allows simulations of dynamical systems in non-real time (simulated mode). There are several component libraries (block sets) that can be interconnected in easy and intuitive ways, similar to a block diagram.

In particular, there are components that allow to simulate dynamic systems, signal generators, signal viewers and communicate with various I/O card, including the ones to be used in this work.

However, since Simulink by itself does not allow to carry out simulations with a synchronously real clock, it is necessary to rely on blocks that interface with A/D and D/A converters. See the user manual of Simulink, available on the website of the discipline.

For a good understanding of the process of creating code in real time it is recommended

to consult the user manual, available on the website of the discipline. We recommend in particular reading the chapter “Basic Procedures”. Nevertheless, it is remarked that reading these documents is not essential to perform this project.



Tasks to be performed in sessions L2–L3

Number of 1h30 laboratory sessions needed to complete this task: 2 (L2 and L3).

Session L1 is not a working session: it is used to form the groups, to go through the safety warnings at the start of this guide, and to walk through the structure of the work. No equipment is operated.

Objectives of L2: at the end of the session you should have a Simulink model that generates a signal, records it in the MATLAB workspace, and drives the motor through the **D/A converter in real time**, together with the observed behaviour of the motor under a constant command.

Objectives of L3: at the end of the session the following elements should be available:

1. SIMULINK block diagram to impose command from the motor and to read signals from the plant;
2. Numerical values of the constants that relate the electrical tensions read at the A/D converters with the corresponding physical variables (motor shaft rotation angle and rotation of the tip of the bar), that is, K_p and K_e , together with the raw calibration data from which they were computed.

Homework to prepare L2

1. **Read Sections 1-3 of this lab guide.**
2. Review your knowledge of basics with MATLAB and SIMULINK. You may find it useful to **follow one of the many excellent tutorials found in the Web.**
3. Work through steps 1–7 of Section 3.2 in simulation on your own machine. These steps need no laboratory hardware, and doing them at the bench would consume most of L2.

Homework to prepare L3

1. Read Sections 3.4 and 3.5, and decide in advance which of the two calibration approaches of step 18 your group will follow.
2. Prepare a table in which to record the deflection of the comb, in slots, against the measured tension, so that the readings of K_e can be taken quickly at the bench.

The distribution of the steps of Section 3 over the two sessions is: L2 covers steps 1–13 (Sections 3.2 and 3.3); L3 covers steps 14–19 (Sections 3.4 and 3.5).

This introductory block is formatted as a tutorial. As such, few previous knowledge of MATLAB/Simulink will be required. All the necessary steps to configure the simulations are quite detailed.

Since some of the procedures are repeated several times throughout the work, we strongly advise the students to draw up a “check-list” based on the tasks performed, synthesizing the procedures required to configure a real-time simulation, configure the recording of data in the workspace, configure the elements viewers, just to mention a few. With these working procedures, work efficiency will be greatly increased.

3.2 Generation, viewing and recording signals.

1. **Open the SIMULINK.** Open the MATLAB (version 8, **R2015a**) and run the command `<<simulink>>`. The window “Simulink Library Browser” will be opened.
2. **Create a new model.** In the menu “Simulink Library Browser”, choose File-New-Model, or press **CTRL +N**, or click on the Simulink button in the top bar of the MATLAB window. This opens a graphical blanch window where you will enter and connect the various operational blocks by dragging from a menu as explained next.
Attention: if you use your laptop with a later version of MATLAB, the new model must be saved as “MATLAB 2015a” to be opened by the desktops at the lab.
3. **Insert and configure a block signal generator.** In the selection tree to the left of the window “Simulink Library Browser” choose “Simulink-Sources”. Drag an element “Signal Generator”  for the model window. Open the element (double click) and configure it to generate a square wave 2 volt amplitude and frequency 2 Hz.
4. **Insert and configure a block signal Viewer.** In the selection tree choose “SimulinkSinks” and drag an element “Scope” to the model window. Open the Scope block and click the toolbar “Parameters” (second from left). In the tab “General” configure “Tick Labels-All”, “Sampling-Sample Time” and choose as sampling period 1 millisecond (0.001). In the tab “History” remove “Limit date selection points to last” and select “Save data to workspace”. The data presented in this element will be exported to the MATLAB workspace at the end of the simulation. Choose the name of the variable “input” and data format “Structure with time”.
5. **Configure and run the simulation.** Connect the “Signal Generator” element to the element “Scope”. Start the simulation (“Simulation-Start” menu, or **CTRL-T**, or press the button ). Observe the result of the simulation in the viewer element “Scope”
6. **Showing signals in workspace.** Confirm that the workspace of MATLAB has the “input” variable. You can do this with by typing “who” in the MATLAB command line. All the existing variables will be listed. Access the signal with the command “plot” and confirm that this signal has an identical shape to the one observed in the “Scope”:

```
>>plot (input.time, input.signals.values)
```

7. **Save data from the workspace.** Save the data file (see how the command “save” operates with the help command: `>> help save`) in order to generate the graph offline.

3.3 Sending commands to the motor.

8. **Insert a block D/A to send commands to the motor.** In the selection tree select “Simulink Desktop Real-time” and drag the “Analog Output” element to the window you have created for your project.
9. **Configure the block D/A.** To see which data acquisition card is in use at the laboratory bench, run the utility “Measurement & Automation” that you can find on the Desktop. In the directory tree that appears on the left side of the main window, choose “Devices and Interfaces”/”NI-DAQmx Devices”, showing the type of graphics card installed (e.g. NI PCI-6221 or PCI-MIO6040E). Make a note of the type of card. Then go back to Simulink, open the D/A block and select the appropriate board (most probably, the correct one is the only one listed). Set the sampling period (sample time) to 1 millisecond (0.001). Set the output channel 1. Voltage ranges should be configured between -10 and 10 V and the data type as “volts”. The initial and final values must be set to 0.

For safety reasons, it is quite important that you set the final value to 0 (zero). This is the value that remains at the output of the D/A board after the experiment is ended. If the final value is not zero, the motor will start running in an endless way in one direction and the equipment may be damaged.

Connect the signal generator to this block.

10. **Configure the simulation to work in real-time.**
 - (a) In the menu “Simulation,” select “Configuration Parameters” or press **CTRL-e**. This opens a dialog window.
 - (b) In the Solver option, go to **Type** and choose “Fixed-step”.
 - (c) In “Code Generation,” set the System Target File by browsing to choose `rtwin.tlc`.
 - (d) In “Hardware Implementation,” set:
 - Device vendor: Intel
 - Device type: X86/Pentium
11. **Configure Simulink external mode.** In the menu “Code” choose “External Mode Control Panel”. This opens a dialog window. Select “Signal & Triggering”. This opens a new window. In “Duration”, enter the number of total samples required for the simulation, in this case $10\text{sec} \times 1000\text{samples/sec} + \text{time instant zero } (t_0) = 10001$ samples. Check that simulation mode is “External” by looking at the Simulink command bar (see Figure 9). If not, go back to the model window and, in the menu “Simulation”, select “External”. Understand the impact of increasing or decreasing the sampling time.



Figure 9: Simulink command bar.

12. **Save the project to disk.** Save the project to a file in the disk (e.g., at the desktop) by using the group “file” in the Simulink task bar. Remember also to record your work to a personal mass storage device, such as a pen that you carry with you. Please clean the files you produced at the end of the lab session.
13. **Make an experiment of sending command signals to the motor.**
Ask the laboratory staff to check whether everything is correct and, after the “go-head” from the staff, start the operation by pressing the button . When needed, use the button  to stop. Observe the motor operation and register its behavior.

3.4 Reading signals from the sensors.

14. **Insert A/D block to read the values of the sensors.** In the selection tree select “Simulink desktop real-time” and drag one block “Analog Input” to the project template.
15. **Configuring the A/D block.** Open the block and choose the acquisition card installed in the machine. Set the sampling period (sample time) to 1 millisecond (0.001 seconds). Set the input channel 1 (potentiometer) to the block. Voltage ranges should be configured between -10 and 10 V and the data type as “volts”.
16. **Insert one block “Scope” to show the signal of the sensor.** Configure it as in point 4. Give it the name “tensao_pot”.
17. **Run the experiment.** Confirm with the teacher that everything is well before beginning the experiment. Observe and register the graphics of the time evolution of the potentiometer voltage. Make sure that the vector “tensao_pot” is available in the workspace of MATLAB and save it to a file.

3.5 Sensor Calibration.

18. **Calibrate the potentiometer sensor model.** This calibration must be performed without the bar attached to the motor. Two types of approaches can be followed: i) applying a voltage to the motor or ii) rotating the gear manually. In i) apply a step signal (1 volt) to the motor and observe the sawtooth wave of the signal obtained by the potentiometer. Discover how to use this information to perform the calibration. In ii) use your imagination, but without damaging the equipment. Select one approach or a combination of both, acquire and record the signals obtained by the potentiometer. From the data obtained, calculate the calibration factor K_p (degree/volt), that make it possible to convert sensor measurements in volt for degrees.
19. **Calibrate the bar deflection model.** Similarly, make a set of experiments to estimate the calibration factor K_e (degree/volt) for the bar deflection. Use the

procedure described in relation to Figure 6 and Figure 7. In this calibration, the cable shall be connected to the strain gage and the bar is outside of the motor shaft but installed on the rig.

An example of a Simulink model for sending commands to the motor and reading signals from the sensors is depicted in Figure 10.

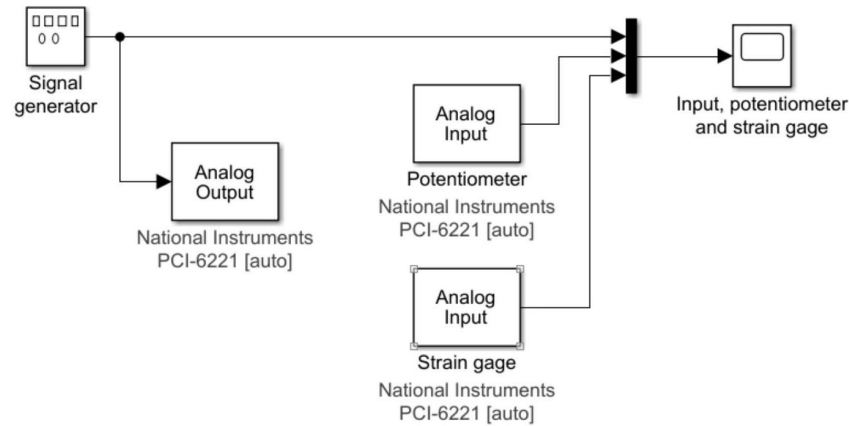


Figure 10: An example of a Simulink model for sending commands to the motor and reading signals from the sensors.



Part I – Q2 Interfacing the plant with the computer

This part of the report must include:

1. Comment on the motor operation. Explain in particular why their angular position never stabilizes when its command signal is constant in the open loop conditions described.
2. Explain the procedure carried out to perform sensor calibration. Include in the report:
 - The Simulink block diagram used;
 - Plots of the sensor and command signals (don't forget to label the variables used in each axis and the units used!);
 - The auxiliary calculations and/or the graphics used to compute the conversion factors K_p (degree/volt) and K_e (degree/volt).
 - Plot on the same graphic the experimental points obtained for the bar deflection and the linear relation assumed to hold between the bar deflection and the sensor electric tension for the value of K_e that you have estimated.

Important: When writing your report, do not forget to display the units in both axes of graphs and to write appropriate captions to identify all the graphs and diagrams, explaining the experimental conditions to which they refer. ■

4 Plant model identification

Plant model identification aims at producing a model to be used for control design. For that sake, the following steps are performed:

1. Perform an experiment to collect data;
2. Process the data to remove undesirable components (for instance, to remove an offset or to filter high-frequency noise);
3. From the processed data identify a model in the form of a difference equation (an ARMAX model). This task comprises:
 - a. Select (following a trial-and-error method) the best model orders (number of poles and zeros);
 - b. Estimate the model parameters
4. Convert the ARMAX model to an equivalent state model.

Data collection is performed using the Simulink block diagrams build as explained in the previous lab session (see Section 3).

A key aspect for the success of your work consists of the experimental conditions under which data is obtained. In this respect the following issues are quite important:

- There is a trade-off in the selection of the amplitude of the signal applied to excite the motor.
 - The amplitude cannot be too low because otherwise the gearbox backlash (folga da caixa de desmultiplicação) prevent the bar to move;
 - The amplitude cannot be too high because there may be nonlinearity effects that distort the signal with respect to linear behavior.

We suggest 1V for the amplitude or values around it.

- The duration of the experiment may not be too long because, since the electrical tension applied to the motor is not perfectly symmetrical, the bar has a tendency to rotate with a drift to one side, dragging the cable connected to the strain-gage, a fact that must never occur. We suggest a few tens (algumas dezenas) of seconds for the duration of an experiment. We also suggest that the initial points are discarded, for instance the first 5 or 10 seconds.
- If you excite the motor with a square wave, you should record the data corresponding to several cycles (at least 2 or 3) of this signal. The exciting signal bandwidth must be carefully selected: The exciting signal may not be too fast (the plant filters it and is not able to respond in the frequency range where its dynamics is dominant) and may not as well be too slow (since the transients are not excited). We suggest frequencies for the excitation signal of about 0,4 Hz. Consider different frequencies and different types of waveforms for the excitation signal, in addition to the square wave (for instance a pseudorandom binary sequence - PRBS). You can generate a PRBS signal in MATLAB with the command `idinput`
- A major issue is the value of the sampling period. Use $20 \text{ ms} = 0,020 \text{ s}$, that corresponds to a sampling rate of 50 Hz .

It is suggested that the data of the different experiments performed is placed in files of type `.mat`. You can organize your software in three different MATLAB `.m` script files that perform the required tasks as follows:

- **Script for model identification.** Data processing, ARMAX model identification from plant data, and conversion to state model is performed in one file (of type `.m`). This file starts by reading plant data stored in a `.mat` file and produces as output the matrices that define the state model that become available in the MATLAB Workspace.
- **Script for controller design.** Another `.m` script file uses these matrices to design the gains of a state feedback controller.
- **Script for controller testing.** A third script uses the matrices that define the state model and the controller gains to parameterize and call a Simulink diagram connected to the plant to perform the tests in closed loop.

Hereafter, we describe a way to encode the first script (for identification). You may use the lines of code suggested in a `.m` file of your own to identify the model from plant data.

It is assumed that the data (obtained by performing plant experiments and following the data acquisition procedure described in Section 3 of this guide) is stored in a data structure generated by a SIMULINK block of type scope. By default, the data is thus stored in a data structure named `ScopeData` (the name of this variable may be changed in the scope menu, tab "History"; actually, in the first lab session you were told to use the name "input") and may be accessed by the MATLAB commands

```
1 t = ScopeData.time;  
2 sigs = ScopeData.signals.values;
```

In the above code, `t` is a vector that contains as entries the sampling instants in continuous time units (s), and `sigs` is a matrix with 3 columns, each corresponding to a variable, and a number of rows that is equal to the number of data points:

- The first column contains the samples of the exciting signal applied to the motor;
- The second column contains the signal issue by the potentiometer that yields the motor shaft angle;
- The third column contains the strain gage measurements.

Off course, you may change the column orders. To use variables with more compact and appealing names you may use the code:

```
1 utrend = sigs(:,1); % Input signal  
2 thetae = sigs(:,2); % Potentiometer signal  
3 alphae = sigs(:,3); % Starin gage signal
```

The position of the bar tip may then be computed using:

```
1 %Computation of total bar angle  
2 ytrend = thetae*Kp + alphae*Ke;
```

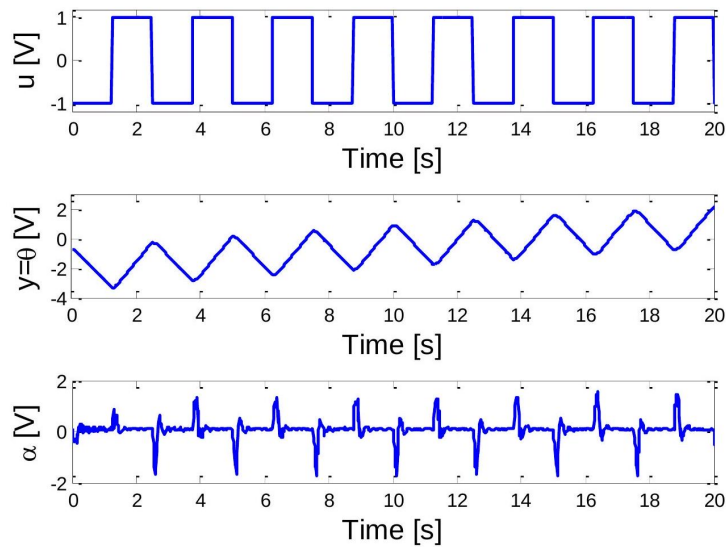


Figure 11: Example of the signals registered in one trial.

An example of a record of the above signals may be seen on Figure 11. It is remarked that the signal that corresponds to the motor shaft angle steadily grows in time. This is due to the fact that the input tension signal has a non-zero average value, and there is an integrator in the motor, given by the relation between the angular speed and the angular position of the motor shaft. The tension created by the cable that connects to the strain gage has a strong impact in this behavior and, thus, the cable should be carefully placed to reduce its perturbation. Nevertheless, in order to get good identification results, this integral effect must be removed before applying the identification methods described hereafter.

Therefore, to build a model for the motor shaft angle dynamics, one must do the following:

1. Remove the integral effect from the motor shaft angle data;
2. Identify the model that relates motor electrical excitation with the motor shaft angle;
3. Add an integrator to the model (pole at 1).

Step 1 above is performed by differentiating the motor shaft angle data. Since differentiation increases high frequency noise (Why? Think in terms of the transfer function of the derivative; for simplicity, consider the continuous time case) a low pass filter must be added to attenuate high frequency noise.

Both operations (differentiation and filtering) may be done with the following MATLAB code:

```

1 af = 0.8;
2 Afilt = [1 -af];
3 Bfilt = (1-af)*[1 -1];
4 % Filtering and detrending
5 yf = filter(Bfilt,Afilt,ytrend);

```

It is a good exercise to write the mathematical expression of the filter and to see from its poles and zeros what are the mathematical operations that this filter performs on data.

In addition to the above code, one should remove the `tref` (average value) of the input signal. It is also useful to remove the trend from the output signal. This can be done with the function `dtrend`.

```
1 u = detrend(utrend);
```

After performing the above operations,  have a pair of input/output signals that can be used to identify a model for the signal (motor plus bar). To perform identification, you may use the function `armax` of Control Systems Toolbox (MATLAB), according to the code:

```
1 z = [yf u];
2 na = 3; % AR part
3 nb = 2; % X part
4 nc = na; % MA part
5 nk = 1; % pure delay
6 nn = [na nb nc nk];
7 th = armax(z,nn) % this is a structure in identification toolbox
    format
```

If you simply write `th` at the MATLAB command line, the content of this structure is echoed in the form of the model. However, for the sake of programming, you must extract from the vectors of the coefficients of the numerator and denominator polynomials of the model. This can be done with the command

```
1 [den1,num1] = polydata(th);
```

Before adding the integrator and build the state model, you can validate the model you have obtained by comparing the output response of the model to the input signal with the differenced and filtered data of the real system. You may simulate the model with the function `filter`:

```
1 yfsim $=$ filter(num1,den1,u); \% Equivalent to idsim(u,th);
```

Figure 12 shows this comparison (this example does not necessarily uses the orders used in the code above). Try with different orders in order to obtain a model with an order that is not too big, but that leads to a good fitting. Observe that the answer to the question *What is the best order?* Is given in ultimate terms by the performance of the controller you will design with the model in a later stage of this work!

You should now add the integrator that was removed from data. This may be done with the function `conv` (test this function separately to multiply to 1st order polynomials and check the result):

```
1 [num, den] = eqtflength(num1, conv(den1, [1 -1]));
```

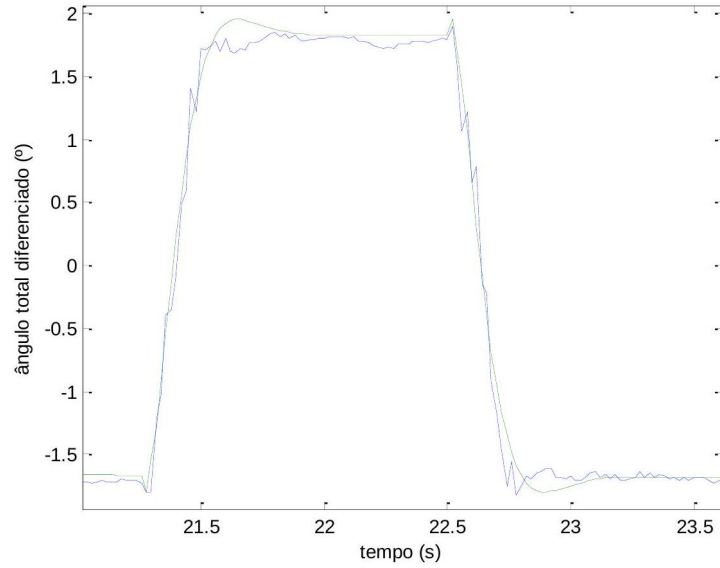


Figure 12: Comparison of the data for the differentiated output with the model output.

Observation: It is remarked that model identification was performed with the System Identification Toolbox, that assumes **s that the polynomials are written in the delay operator**. However, conversion to the state model and model design are performed with the Control Systems Toolbox, that assumes that the polynomials are written in the forward shift operator. In order to convert one polynomial representation into another we used above the function **eqtflength**.

At this point, vectors num and den have the coefficients of the numerator and denominator polynomials of the transfer function of the model identified for the relation between the electrical excitation of the motor and the bar tip. These polynomials assume a parametrization in the forward shift operator. We can now use the function

```
1 [A, B, C, D] = tf2ss (num, den);
```

to make the conversion to the state model parametrized by

$$x(k+1) = Ax(k) + Bu(k) \quad (4.1)$$

$$y(k) = Cx(k) + Du(k) \quad (4.2)$$

where u (scalar) is the motor excitation, y (scalar) is the flexible bar tip angular position, x ($n \times 1$) is a column vector and A ($n \times n$), B ($n \times 1$), C ($1 \times n$) and $D = 0$ (scalar) are matrices of compatible dimensions, n being the dimension of x . The index k denotes discrete time.

These matrices form the input to the next phase of the project, concerned with controller design.



Tasks to be performed in sessions L4–L7

Number of 1h30 laboratory sessions: 4 (L4 to L7).

Before the first of these sessions, to prepare them: Read carefully Section 3 of this guide. Write a MATLAB script to test, correct and use during the lab sessions, that embeds the procedure and code described above. You will probably need to add some lines of MATLAB code to define some variables (for instance, the sampling interval) or make plots.

It is quite important that, before L4, you do this preparation. With 1h30 per session, a group that arrives at L5 without a working identification script will not have time to write one and identify a model in the same session.

L4 – Collection of identification data

Objective: at the end of the session you should have at least two recorded datasets, obtained **with different excitation signals, stored in .mat files and known to be usable.**

1. Select an input signal (for instance a square wave or a PRBS) and perform an experiment in which the plant is excited with it. Register the data. For this sake, use the data acquisition facilities of SIMULINK that you have learned how to use in Section 3, and write an appropriate SIMULINK block diagram. Use a sampling interval of 0,02s (sampling frequency of 50Hz). Observe that the motor + gear box has a dead zone.
2. Repeat the experiment with a second waveform and with a different amplitude, so that in the report you are able to discuss the effect of very small and very large amplitude excitation signals. **Record a further dataset that you will keep aside for validation only, and do not use it for identification.**
3. Before leaving the bench, **plot each dataset and check that the three signals look like** those of Figure 11. **Data that turns out to be unusable cannot be recollected in L5.**

L5 – Data processing and ARMAX identification

Objective: at the end of the session you should have an ARMAX model of the plant and a justified choice of model orders. This session needs no plant, so it can also be used to catch up on any data that is missing from L4.

1. The plant output (angular position of the flexible bar tip) data has to be treated in order for the identification algorithm to work properly
 - (a) Remove **constant offsets in data using the MATLAB function detrend**
 - (b) Differentiate the plant output to remove the integrator associated to the motor and filter the output to eliminate high frequency dynamics and noise that fall outside the frequency band in which the model is to be valid. For that sake use the commands suggested above.
2. Identify an **ARMAX model using the function armax;**
3. Explore several combinations of n_a , n_b and n_c in a critical way, and record the fit obtained for each, so that the choice of orders can be justified in the report.

L6 – State-space model and validation

Objective: at the end of the session you should have the matrices A , B , C , D of the state model, validated against data that was not used to identify it. These matrices are the input to the whole of Section 5.

1. Add the integrator to the model (use the function `conv`);
2. Convert the model to state-space form using the function `tf2ss`;
3. Validate the model in various forms.
 - (a) Compare data that has not been used for identification with the result obtained by simulating with the model using as input the same input used to excite the system;
 - (b) Look at the simulated time response and discuss its plausibility;
 - (c) Look at the frequency response and discuss its plausibility.

L7 – Review of the identification

Objective: the model that goes into Part II, together with the evidence that it is the best of those you tried. This session closes Part I.

1. Review the process, changing the values of some parameters (filter pole, assumed model orders, sampling interval), and compare the resulting models on the validation dataset.
2. If any of the data recorded in L4 turned out to be unusable, this is the last session in which it can be recollected.
3. Fix the matrices A , B , C , D that will be used throughout Section 5, and record the open-loop pole-zero plot, frequency response and time response that Q3 asks you to characterise.



Part I – Q3 The plant model

This part of the report must address the following issues:

- Explanation on the tests performed on the plant to obtain the data used for identification. Discuss the results obtained with different types of excitation signals and the effect of very small amplitude and very large amplitude excitation signals;
- Discussion of the sampling frequency;
- Effect on identification of filtering the data;
- Explanation on how the pole at the origin has been dwelt with;
- Discussion on how the model orders have been decided. Take into consideration that the plant results from the interconnection of a DC motor and the flexible bar and discuss the models needed for each of them;
- Description of the final ARMAX model;
- Description of the final state-space model.
- Characterization of the plant open loop pole-zero plot, frequency response and time response of the model.
- Model validation;



5 Controller design

In this section, the state-space model identified described on Section 4 is used to design a LQG controller.

The controller consists of a state feedback control law whose gains are selected such as to minimize a quadratic cost. The controller gains are designed on the basis of the state model (4.1), (4.2) that has been identified. This is the LQ control law.

Since the state of the plant is not available for direct measurement, it is estimated with a Kalman filter. The Kalman filter receives as input the plant input and output and yields as its output an estimate of the plant state. The Kalman filter has a structure that is equal to the current observer, but its gain is selected in a particular way so as to optimize the signal-to-noise ratio of the estimate in the presence of noise that affects the plant.

The equations of the algorithm that are required to perform this work are described hereafter. Further details can be seen in the book:

G. F. Franklin, J. D. Powell and M. Workman. Digital Control of Dynamic Systems. 3rd ed., Addison Wesley, 1998, chapter 9 “Multivariable and Optimal Control”.

5.1 LQ control law

The control law that generates the manipulated variable u for the state model (4.1), (4.2) such as to minimize the steady-state (or infinite horizon) quadratic cost

$$J = \sum_{k=1}^{\infty} x^T(k)Qx(k) + u^T(k)Ru(k), \quad (5.1)$$

where $Q \succcurlyeq 0$ (means Q is positive semi-definite) and $R \succ 0$ (means R is positive definite) is given by the state feedback

$$u(k) = -Kx(k), \quad (5.2)$$

where the vector of feedback gains K (with dimension $1 \times n$) is computed by

$$K = (B^T S B + R)^{-1} B^T S A. \quad (5.3)$$

The matrix S ($n \times n$) is the only positive definite solution of the algebraic Riccati equation (ARE)

$$A^T S A - A^T S B (B^T S B + R)^{-1} B^T S A + Q = 0 \quad (5.4)$$

If there exists a K such that the closed-loop system that results from coupling (4.1) with (5.2) minimizes the cost (5.1), then it can be found by (5.3), (5.4).

For the purposes of this work, The LQ gain is computed using the MATLAB function `dlqr`. Use the MATLAB help to know how to use this function.

The function also computes the closed-loop eigenvalues, given by the eigenvalues of $A - BK$. This result is an important piece of information. For a well posed problem, the real part of these eigenvalues is always strictly negative.

The main “tuning knob” for the designer when using LQ control are the matrices Q and R . For a SISO plant, if the output y is to be regulated, then (due to (4.1) and $D = 0$), we have

$$Q = C^T C. \quad (5.5)$$

The weight R reduces thus to a scalar. Changing R adjusts the type of response yielded. Indeed, the poles of the closed-loop system are the stable roots of the root square locus. In general, decreasing R increases the bandwidth of the control system. Thus, decreasing R has the advantage of making the response of the close loop faster, but the disadvantage of exciting plant modes that are not modeled.

You may get an overall picture of the dependence of the closed-loop poles on R using a technique known as root-square locus. See the companion document or the reference book.

Warning: Do not confuse `dlqr`, that solves the LQ problem in discrete time (the formulation used here) with the function `lqr` that solves a similar problem in continuous time (not considered in this work).

5.2 Kalman filter

The Kalman filter aims at estimating the plant state from the observation of the plant input and output. It assumes that the plant model is modified by the inclusion of “disturbance” or “noise” inputs, becoming

$$x(k+1) = Ax(k) + Bu(k) + w(k), \quad (5.6)$$

$$y(k) = Cx(k) + Du(k) + v(k). \quad (5.7)$$

The signal w is the process noise (that models random disturbances) and the signal v is the sensor noise. The Kalman filter has the structure of the current observer, with the gain optimized by taking into account these noise terms.

The equations of the Kalman filter (current observer) are given by

$$\hat{x}(k | k-1) = A\hat{x}(k-1 | k-1) + Bu(k-1), \quad (5.8)$$

$$\hat{x}(k | k) = \hat{x}(k | k-1) + M(y(k) - C\hat{x}(k | k-1)). \quad (5.9)$$

These equations are interpreted as follows:

- Equation (5.8) computes what you expect the state to be at time k given the previous estimate $\hat{x}(k-1 | k-1)$, the input sample applied, $u(k-1)$, and the knowledge about the state dynamics.
- Then, in (5.9), this prediction is corrected by a term that is proportional to what you expect the output to be given the state prediction, $C\hat{x}(k | k-1)$, and what is actually observed for the value of $y(k)$.

The proportionality constant vector M is called the Kalman gain. It is computed so as to optimize a quadratic cost.

In this work, the Kalman gain is computed using the MATLAB function `dlqe`. Use the MATLAB help to know how to use this function. Again, do not make a confusion with the function `lqe` that computes the Kalman filter gain for the continuous time case.

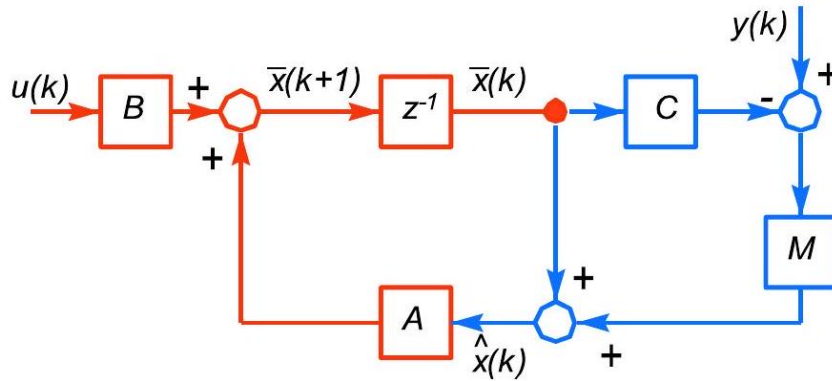


Figure 13: Block diagram of the state estimator. The part in red corresponds to the predictive observer and the part in blue to the correction to obtain the current estimate $\hat{x}(k)$.

Figure 13 shows the block diagram of the current observer (i.e., Kalman filter). Its inputs are the output plant observations, $y(k)$, and the samples of the manipulated variable applied to the plant, $u(k)$.

5.3 Loop transfer recovery (LTR)

The computation of the Kalman gain requires the covariance matrices of the noises entering the model. These are $Q_w = \mathcal{E}(ww^T)$ and $R_v = \mathcal{E}(vv^T)$. In this case, R_v is a scalar. Instead of trying to estimate these noise covariances from plant data, we are going to use them as “tuning knobs”. Thus, we make the choices

$$Q_w = 100I \quad \text{and} \quad R_v = 1.$$

Also for the observer, you may get an overall picture of the dependence of the error equation poles of the observer on R_v using a technique known as root-square locus. See the companion document or the reference book.

5.4 LQG controller structure

The LQG controller is obtained by coupling a LQ controller with a Kalman filter and replacing the state by its estimate, that is to say, replacing (5.2) by

$$u(k) = -K\hat{x}(k).$$

This controller has the structure shown in Figure 14.

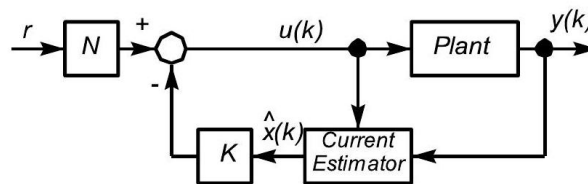


Figure 14: Block diagram of the controlled system.

The constant N is such that, in closed-loop, the gain from the reference r to the output y is equal to 1. For a state of dimension 5 it can be computed by the MATLAB code:

```
1 N = inv([A-eye(5,5), B; C,0])*[zeros(5,1);1];
2 Nx = N(1:5,:);
3 Nu = N(6,:);
4 Nbar = Nu+K*Nx;
```

See the reference book on page 313 for a justification of this formula.



Tasks to be performed in sessions L8–L12

Number of 1h30 laboratory sessions: 5 (L8 to L12).

Before the first of these sessions, to prepare them: Read carefully Section 4. When doing this, draw (using pencil and paper) a detailed block diagram of the controller by inserting Figure 13 and Figure 14. There is a companion text to this guide that summarizes the main point of state feedback design. It is suggested that you study it carefully before the lab session. Write a MATLAB script to design and test the controller to test, correct and use during the lab sessions, that embeds the procedure and code

described above. It is advisable that you test the controller design on the model that has been identified in Section 4.

It is quite important that, before L8, you do this preparation.

Note that L8 and L9 do not require the plant: the design and the simulation are done on the model fixed in L7. Only from L10 onwards is bench time indispensable. A group that treats L8 and L9 as homework and arrives at L10 with a working design will have three sessions of plant time instead of two, which in practice is the difference between a tuned controller and an untuned one.

L8 – LQ design on the identified model

Objective: a state feedback gain K and an understanding of how it depends on R .

1. Compute K with `dlqr` using $Q = C^\top C$. Start by making $R = 100$.
2. Examine the closed-loop eigenvalues of $A - BK$ and build the root-square locus in R .
3. Simulate the state feedback on the identified model and decide what features of the time response you will call “satisfactory”.

L9 – Kalman filter, LTR and closed-loop simulation

Objective: a complete LQG controller, verified in simulation before it is connected to the plant.

1. Compute the Kalman gain M with `dlqe`, starting from $Q_w = 100I$ and $R_v = 1$.
2. Build the observer root-square locus in R_v and observe what happens when R_v is increased.
3. Couple the observer with the state feedback, compute N and $\bar{N}$ as in Section 5.4, and simulate the full LQG loop on the identified model.

L10 – First closed-loop tests on the plant

Objective: the controller running in real time on the real plant.

1. Build the real-time SIMULINK diagram that implements the observer, the feedback gain and the pre-filter, and have the laboratory staff check it before the first run.
2. Run the closed loop on the plant with the design that was satisfactory in simulation, and record the time response.
3. Compare the response obtained on the plant with the one obtained in simulation, and note where they differ.

L11 – Tuning on the plant

Objective: the record of a set of experiments that show that the controller you selected is the best choice.

1. Explore several options by changing the weight matrix R , and record the response obtained for each.
2. Repeat for R_v , and relate what you observe on the plant to the loop transfer recovery argument of Section 5.3.
3. Assess the effect of the pre-filter by running the loop with and without it.

L12 – Final measurements

Objective: everything Section 5.5 asks you to document.

1. Record the final closed-loop time response, the closed-loop frequency response and the loop gain.
2. Check that every figure you intend to include in the report has been recorded with its units and its experimental conditions noted, since the bench will not be available again.
3. The report is due at 23:59 on the day of this session (7 December for the Monday shifts, 10 December for the Thursday shifts).

L13 – Final laboratory assessment

L13 involves no new experimental work. The grade and the feedback on the report are released online between L12 and L13, and there is a window in which you may contact the teaching team to clarify them or to point out anything you consider incorrect or missing in the appraisal of your report. L13 itself is the final laboratory assessment, held in your own shift slot, in which the collective and individual validation questions are put to the group. Attendance is compulsory: an unjustified absence carries a grade of 0 in the laboratory component.

5.5 Documentation

Among other things you find convenient to include in your report, document your design with

- The time response of the closed-loop system;
- The frequency response of the closed-loop system; and
- The loop-gain.



Part II – Q4 The control design

This part of the report must cover controller design and testing of the controlled system, including:

- The MATLAB scripts that must include comments;
- The block diagram used to control the system;
- Comment on the choice of the weights on the quadratic cost when using the LQG design approach. Include the root-square locus;
- Effect of the choice of the noise covariance matrices in a LTR framework;
- Resulting closed-loop frequency response and time response;
- Effect of the inclusion of a pre-filter;
- Discuss how do you evaluate the performance of your control system and what are the limits of performance.



6 Instruction on the report and software to develop

6.1 Report Content and Submission

The students must submit **one single report**, covering the whole of the work and comprising the answers to Q1, Q2, Q3 and Q4, as marked by ✂ and in the highlighted blue boxes. The report should be organised in two parts, reflecting the structure of the work:

1. **Part I – Model identification and validation**, comprising the answers to Q1, Q2 and Q3;
2. **Part II – Control design and testing**, comprising the answer to Q4.

Although the report is submitted only once, at the end of session L12, the material for Part I is complete at the end of session L7. Writing it up at that point, rather than in the final weeks, is strongly advised: L10 to L12 are the sessions that require the plant, and groups that leave the whole write-up to the end invariably lose bench time to it.

The symbol  corresponds to tasks to be performed during the laboratory sessions.

6.2 Language

The language of the report may be either Portuguese or English.

6.3 Report Format

The report must be written using the L^AT_EX template made available on the course website, which already carries the required structure. It must indicate:

- The number and name of the students that author the report, and the laboratory shift the group attends (CSCibL02, CSCibL03, CSCibL04 or CSCibL05).
- The number of the questions addressed as indicated in this guide (Q1, Q2, Q3, and Q4), each under the part to which it belongs — Part I for Q1–Q3, Part II for Q4.

The content of the report, counted from the Introduction to the Conclusions inclusive, may not exceed **15 pages** after compilation to PDF. Anything beyond that is not evaluated.

The following do not count towards that limit: the cover page, the table of contents, the declaration pages, and the appendix containing the code. Use as many declaration pages as you need, and do not shorten or omit code in order to save space.

The appendix must contain the three scripts described in Section 4 — identification, controller design and controller testing — **and every function you created**, whether or not it is called by those three scripts. Use one listing per file, with the file name in the caption, and keep the comments in the code.

The declarations on the use of artificial intelligence and on originality are part of the report and must be completed. The laboratory work is a green (Level 2) component and the writing of the report is a yellow (Level 1) component; both require the declaration to list every tool used and the purpose of each use. Undeclared use constitutes a breach of academic integrity.

6.4 Answer Guidelines

The answers must be direct, concise and short, but insightful and technically correct.

6.5 Submission

Both the report and the software must be sent to the professor in charge of the laboratory within the prescribed time limits. The final submission is due at 23:59 on the day of session L12 (week 14), that is, on 7 December for shifts CSCibL02 and CSCibL03 and on 10 December for shifts CSCibL04 and CSCibL05. The deadline is anchored to each shift's own session so that every group has had exactly the same number of laboratory sessions before submitting.

6.6 Originality and Plagiarism

The report and the software developed must be original and the data used must be actually obtained by the students that sign the report. All forms of plagiarism or copy detected will be punished without contempt, according to Instituto Superior Técnico's and University of Lisbon's regulations, and the Portuguese law.